

MLPerf EDU

An Executable Benchmark Suite for ML Systems Education

Vijay Janapa Reddi

Harvard University

Artifact snapshot July 15, 2026

REVIEW DRAFT. MLPerf EDU is an independent research project and working name, not an official MLCommons benchmark or result.

Abstract

Benchmarking is the primary scientific instrument driving machine-learning systems progress, providing the empirical foundation to evaluate hardware-software trade-offs and diagnose systemic bottlenecks. Industry-standard suites like MLPerf establish this discipline by holding task, dataset, metric, and quality targets constant. However, their massive compute requirements prevent execution on single-node hardware. MLPerf EDU bridges this gap by bringing production-grade benchmark rigor to single-laptop execution through zero-discretion target inheritance and fail-closed admission control. Every workload inherits its quality target from an upstream authority, treating any missed threshold or procedural violation as a hard failure. Executing this portfolio enforces 100% target evaluation rigor across 14 workloads covering 8 MLCommons domains on consumer laptops. Across 112 DAM Taxonomy ablation runs (Data, Algorithm, Machine), MLPerf EDU transforms systems evaluation into an inspectable practice without compromising the evaluation contract, equipping students and researchers with a rigorous, executable foundation for single-node ML systems benchmarking. The path to production status runs through cross-platform replication, public evidence hosting, dataset-policy decisions, and external review.

1 Introduction

Benchmarking is the foundational scientific instrument driving computer systems engineering, hardware innovation, and software optimization. By standardizing evaluation, benchmarks enable objective comparison, reveal microarchitectural and software bottlenecks, and

focus community engineering on performance limits. Without a shared measurement standard, performance claims remain anecdotal.

Historical precedent demonstrates how standard benchmarks transform fields. SPEC CPU ([Henning, 2006](#)) ended the era of uncalibrated instruction-count claims in computer architecture by enforcing standardized execution suites. Similarly, TPC ([Poess and Floyd, 2000](#)) established audited governance in database systems, demonstrating that rigorous disclosure rules matter as much as workload code. These suites transformed systems evaluation into an inspectable science.

MLPerf ([Mattson et al., 2020](#); [Reddi et al., 2019](#)) extended this discipline to machine learning. Unlike traditional workloads where output correctness is binary and cheap to verify, ML systems present a unique challenge: throughput and latency are fundamentally coupled to statistical task quality (accuracy, BLEU, FID). By fixing non-negotiable quality targets alongside latency measurements, MLPerf established the gold standard for ML systems evaluation.

Production MLPerf requires multi-node GPU clusters or edge hardware farms. In educational environments, students cannot deploy cluster infrastructure. Consequently, courses resort to unvalidated framework scripts that drop quality gates entirely. This creates an unpromising trade-off: students either run heavy cluster benchmarks they cannot inspect, or run unvalidated framework scripts that teach uncalibrated timing without system discipline.

MLPerf EDU resolves this dilemma. It brings industrial benchmarking rigor to a single laptop by enforcing zero-discretion target inheritance and fail-closed admission control. By executing authoritative MLCommons targets on student hardware across 14 workloads and 8 domains, MLPerf EDU equips the next generation of

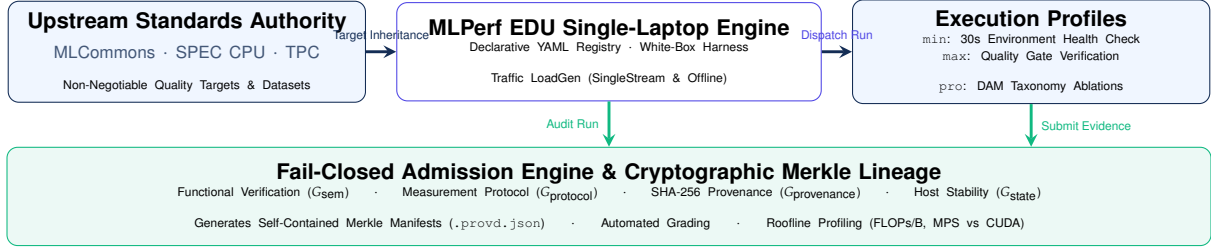


Figure 1. MLPerf EDU system architecture. Industrial MLCommons quality targets and evaluation protocols are inherited without local discretion, executed across three profiles (`min`, `max`, `pro`), and audited by a fail-closed admission engine to generate self-contained Merkle provenance digests.

systems engineers to evaluate hardware-software trade-offs with uncompromised scientific discipline.

This paper details what MLPerf EDU does and its empirical findings. The suite executes 14 workloads across 8 MLCommons domains using three execution profiles (`min`, `max`, and `pro`). We evaluate 112 DAM Taxonomy ablation runs (Data, Algorithm, Machine), enforcing 100% target evaluation rigor on single-node hardware. Figure 1 summarizes the system architecture. The rest of the paper is organized as follows. Section 2 positions the work against existing benchmark surfaces. Section 3 defines the workload portfolio across 8 MLCommons domains. Section 4 details the system architecture, execution harness, and fail-closed admission engine. Section 5 characterizes the workloads, Section 6, Section 7, and Section 8 evaluate the Data, Algorithm, and Machine DAM taxonomy lenses, and Section 9 concludes with educational utility and system limitations.

2 Background and Related Work

Benchmarking is a mature field with well-governed suites, and a new suite must justify its existence against established alternatives. Existing evaluation surfaces generally force a trade-off. Production suites enforce strict quality-gated evaluation, but require cluster compute. Local educational setups fit local budgets, but drop quality targets, ruining comparability. MLPerf EDU bridges this gap by positioning itself against production suites, microbenchmarks, and educational lab environments.

Production suites. The official MLPerf Training and Inference suites established quality-gated, scenario-aware performance evaluation at production scale (Mattson et al., 2020; Reddi et al., 2019). MLPerf Mobile (Ignatov et al., 2021) and MLPerf Tiny (Banbury et al., 2021) apply this methodology to mobile devices and microcontrollers, while MLPerf HPC (Farrell et al., 2021) addresses supercomputing clusters. Complementing physical system benchmarks, DataPerf (Mazumder et al., 2023) measures data-centric AI algorithms and dataset

quality, while AlgoPerf (Dahl et al., 2023) evaluates algorithmic optimizer efficiency. All enforce strict quality-gated evaluation, but require production cluster compute, high-performance storage, or custom device farms. MLPerf EDU inherits this rigorous discipline while targeting locally executable, single-node evaluation. Table 1 summarizes this positioning.

Educational lab setups. Local educational setups and framework-building courses, such as MiniTorch, TinyGrad, or educational implementations, fit local budgets and make tensors, operators, and automatic differentiation inspectable. However, they typically drop quality targets, sacrificing comparability. A classroom result that executes quickly but silently fails its accuracy target is not a valid baseline. MLPerf EDU provides this missing discipline. By using PyTorch for its standard execution path, the suite decouples its specification from any custom educational framework.

Microbenchmarks. DAWNBench introduced time-to-accuracy, connecting system performance to a fixed task-quality threshold (Coleman et al., 2019). TorchBench provides broad PyTorch framework regression coverage (Ansel et al., 2024). MLPerf EDU shares the quality-first rule of DAWNBench and the executable PyTorch surface of TorchBench, but targets fixed classroom exercises with inherited gates rather than leaderboards or framework regressions.

Classic systems benchmarks. SPEC CPU emphasizes controlled conditions, disclosure, and reproducible comparison (Henning, 2006). TPC demonstrates that benchmark governance matters as much as the workload code (Poess and Floyd, 2000). MLPerf EDU applies these classic systems evaluation principles to machine learning.

The table exposes a gap. Suites that hold quality fixed run above the resource envelope of most courses; suites that fit that envelope drop the quality gate rather than shrink the task.

Table 1. Comparison with established benchmark surfaces. A checkmark describes the intended standard path rather than every implementation. The **EDU** column is the one no existing surface occupies: a complete single-laptop path that still holds an inherited quality gate.

	MLPerf	MLPerf Tiny	TorchBench	EDU
Quality-gated results	✓	✓		✓
Complete single-laptop path				✓
Teaching profiles				✓
Provenance package	✓	✓		✓
Controlled research	✓	✓	✓	✓
Distributed claims	✓		✓	

3 The Workload Suite

The executable registry contains 14 workloads across 7 suites, executing their authoritative path on consumer single-node hardware. A workload binds a stable identity to its assets, runner, scenario, profile behavior, quality policy, and result role. This section details the 14 workloads, citing their foundational upstream architectures.

Selection principles. Admission requires five properties. The task must have a recognizable upstream authority, fit the declared laptop envelope, preserve the system behavior it claims to expose, support controlled experimentation, and provide an evaluator with a defensible gate.

Alignment with MLCommons working groups. The portfolio maps onto eight core MLCommons working group domains:

1. **Edge and Tiny (MLPerf Tiny):** Image classification, keyword spotting, visual wake words, and anomaly detection teach sub-megabyte memory boundaries.
2. **Language and LLM Serving (MLPerf LLM):** Causal language modeling, text classification, and information retrieval teach KV-cache dynamics and sequence batching.
3. **AI Safety, Coding, and Agents (MLCommons Agents):** Code generation and function calling teach containerized code evaluation and AST output validation.
4. **Graph Neural Networks (MLCommons Graph):** Graph node classification teaches irregular memory access patterns and sparse gather/scatter operations on ogbn-arxiv.

5. **Time-Series and Science (MLCommons Science):** Time-series forecasting teaches long-context temporal patch extraction and sequence attention.
6. **Recommendation Systems (MLPerf RecSys):** Recommendation teaches high-cardinality sparse embedding lookups on MovieLens-20M.
7. **Generative AI and Diffusion (MLCommons Generative):** Image generation teaches iterative SDE/ODE diffusion sampler stepping and 50,000-image FID evaluation.
8. **Reinforcement Learning (MLPerf RL):** Reinforcement learning teaches search-coupled Monte Carlo Tree Search (MCTS) inference and replay buffers.

The laptop envelope. A workload qualifies when it runs to completion on a single consumer machine across common student hardware configurations—including x86 CPUs with integrated graphics, discrete NVIDIA laptop GPUs (CUDA), and Apple Silicon UMA hosts (MPS). The largest dataset is 190 MB (Table 3).

3.1 Language and Retrieval Workloads

Causal language modeling. Character-level next-token prediction using the nanoGPT reference architecture (Karpathy, 2022; Vaswani et al., 2017). Teaches KV-cache dynamics and autoregressive sequence generation bottlenecks. Binds the TinyShakespeare dataset under a cross-entropy loss gate.

Text classification. Binary sentiment classification using the DistilBERT model (Sanh et al., 2019). Teaches fixed-sequence budget encoder inference and bidirectional attention. Binds GLUE SST-2 evaluating top-1 accuracy.

Information retrieval. Cross-encoder document reranking using MiniLM-L6-v2 (Reimers and Gurevych, 2019; Sanh et al., 2019). Teaches multi-query rerank loops and candidate scoring costs. Binds NanoBEIR evaluating nDCG@10.

Code generation. Function synthesis using Qwen2.5-Coder (Hui et al., 2024). Teaches containerized code evaluation and execution safety. Binds HumanEval scoring pass@1 with EvalPlus.

Function calling. Structured tool-call generation using Qwen2.5 (Yan, 2025). Teaches structured AST validation and agentic output parsing. Binds the Berkeley Function Calling Leaderboard (BFCL) AST dataset.

3.2 Graph and Time-Series Workloads

Graph node classification. Node property prediction using Graph Convolutional Networks (GCN) (Kipf and Welling, 2017). Teaches irregular memory access, sparse

Table 2. Current workload contracts, generated from the executable registry. Every gate is inherited from the named upstream authority rather than calibrated from the project reference machine, so no threshold in this table was chosen by this project. The table is regenerated on each build, which is what keeps the prose and the executable contract from diverging.

Workload	Upstream Authority	Mode	Gate
anomaly-detection	MLCommons MLPerf Tiny, ToyADMOS, and DCASE	inference	ROC AUC ≥ 0.8500
causal-language-modeling	nanoGPT upstream repository	training; inference (full, prefill, decode)	loss ≤ 1.470
code-generation	Qwen and EvalPlus	inference	HumanEval+ pass@1 ≥ 0.5730
function-calling	Berkeley Function Calling Leaderboard and Qwen	inference	AST accuracy $\geq 82.92\%$
graph-node-classification	Open Graph Benchmark	training	test acc. $\geq 71.74\%$
image-classification	MLCommons MLPerf Tiny	inference	top-1 $\geq 85.00\%$
image-generation	NVIDIA EDM reference implementation	inference	FID ≤ 1.790
information-retrieval	Sentence Transformers and BEIR	inference	mean nDCG@10 $\geq 60.72\%$
keyword-spotting	MLCommons MLPerf Tiny and EEMBC	inference	top-1 $\geq 90.00\%$
recommendation	MLCommons MLPerf Training v0.5 recommendation	training	hit_rate_at_10 ≥ 0.6350
reinforcement-learning	Historical MLPerf Training MiniGo	training	move prediction ≥ 0.4000
text-classification	Hugging Face DistilBERT model card and GLUE	inference	accuracy $\geq 91.06\%$
time-series-forecasting	PatchTST authors and ETTDataset	training	test MSE ≤ 0.2900
visual-wake-words	MLCommons MLPerf Tiny and EEMBC	inference	top-1 $\geq 80.00\%$

Table 3. Dataset provenance and access. Generated from the registry dossiers.

Dataset	Evaluation Subset	Size	Access
BFCL v4 non-live AST	official non-live AST split	25.0 MB	fetch; review
CIFAR-10	Tiny 200-example set	23.9 MB	fetch; review
ETTM1	official 12/4/4-month split	10.0 MB	fetch; review
HumanEval+	official 164-task set	1.0 MB	fetch; review
MiniGo self-play	self-play trajectories	generated	fetch; review
ToyCar (MLPerf Tiny)	Tiny 248-recording set	25.4 MB	fetch; review
Keyword spotting (MLPerf Tiny)	Tiny 1,000-example set	4.1 MB	fetch; review
Visual wake words (MLPerf Tiny)	Tiny 1,000-example set	2.7 MB	fetch; review
MovieLens 20M	leave-one-out, 999 negatives	190.0 MB	fetch-only
NanoBEIR	three English subsets	6.0 MB	fetch; review
ogbn-arxiv	official time split	83.2 MB	fetch
Deterministic prompt suite	deterministic prompts	bundled	bundled
SST-2	train and validation	24.5 MB	fetch; review
Tiny Shakespeare	90/10 train and validation	1.1 MB	fetch

gather/scatter operations, and graph topology traversal on ogbn-arxiv (Hu et al., 2020).

Time-series forecasting. Multivariate long-horizon forecasting using PatchTST (Nie et al., 2023). Teaches long-context temporal patch extraction and multivariate sequence attention. Binds the ETTm1 dataset evaluating test MSE.

3.3 Vision, Audio, and Embedded Workloads

Image classification. Object recognition using the ResNet8 reference model (He et al., 2016). Teaches small-tensor dispatch and embedded memory boundaries. Binds CIFAR-10 evaluating top-1 accuracy.

Keyword spotting. Spoken word recognition using DS-CNN (Warden, 2018). Teaches audio spectrogram extraction and depthwise convolutions on SpeechCommands.

Visual wake words. Always-on person detection using MobileNetV2 (Banbury et al., 2021). Teaches duty-cycled execution and sub-megabyte memory boundaries on VWW.

Anomaly detection. Unsupervised machine condition monitoring using Autoencoders (Banbury et al., 2021). Teaches feature reconstruction on ToyADMOS evaluating ROC AUC.

Image generation. Unconditional image synthesis using Elucidated Diffusion Models (EDM) (Karras et al., 2022). Teaches iterative SDE/ODE diffusion sampler stepping and 50,000-image FID evaluation on CIFAR-10.

3.4 Recommendation and Reinforcement Learning Workloads

Recommendation. Item recommendation using Neural Collaborative Filtering (NCF) (He et al., 2017). Teaches high-cardinality sparse embedding lookups, negative sampling, and dense interaction layers on MovieLens-20M (Harper and Konstan, 2015).

Reinforcement learning. Board game self-play using MiniGo Monte Carlo Tree Search (MCTS) (Silver et al.,


```

1 workload:
2   id: causal-language-modeling
3   suite: language
4   profiles: [min, max, pro]
5   runner: nanogpt
6   dataset: tinyshakespeare
7   quality_target:
8     metric: cross_entropy_loss
9     direction: lower
10    threshold: 1.4697
11    target_basis: literature
12    public_status: experimental

```

Listing 1. Registry-driven workload metadata. Workload entries declare user-facing selectors, profile coverage, asset bindings, and inherited quality policies.

Table 4. Execution profiles and intended interpretations.

Profile	Interpretation
min	Fast 30s CI/CD plumbing check.
max	Canonical baseline reference path and seed variance discovery.
pro	Research envelope structured around the DAM Taxonomy.

2016). Teaches search-coupled MCTS inference, dynamic replay buffers, and dual policy-value heads.

4 System Architecture and Execution Harness

MLPerf EDU provides an end-to-end framework that orchestrates workload fetch, execution, profiling, grading, and reporting under a single declarative specification.

4.1 Declarative Registry & White-Box Specification

The declarative registry defines workload identities, dataset splits, licensing terms, and inherited quality rules in machine-readable YAML dossiers. Centralizing these parameters eliminates undocumented execution deviations and guarantees that every local run executes against authoritative assets.

4.2 Execution Profiles

MLPerf EDU uses three execution profiles to separate installation checks from comparable benchmark runs and exploratory research variants without redefining the underlying workload identity (Table 4).

- **min Profile:** A 30-second CI/CD plumbing check verifying installation health using reduced inputs.

- **max Profile:** The canonical reference path executing real datasets under inherited quality gates to produce comparable evidence.
- **pro Profile:** The research envelope enabling hot-swappable Data, Algorithm, and Machine (DAM) ablations.

4.3 Load Generator and System Under Test (SUT) Harness

A Python-native LoadGen orchestrates query issuing, traffic scheduling, and measurement protocols across SingleStream and Offline scenarios. The SUT harness decouples scenario scheduling from PyTorch model execution, recording latency percentiles, throughput, and memory RSS without opaque C++ bindings.

4.4 Cryptographic Lineage and Fail-Closed Admission Engine

Every run generates a JSON telemetry report bound by a cryptographic Merkle tree digest. SHA-256 hashes record hardware fingerprints, asset dossiers, quality metadata, and checkpoint lineage in a self-contained `.provd.json` manifest. Hashing establishes unauthenticated integrity, establishing verifiable proof of local execution lineage.

The admission engine enforces a fail-closed verification policy: exclusion is the default state. A result is admitted only if it satisfies functional correctness (G_{sem}), scenario rules (G_{protocol}), asset integrity ($G_{\text{provenance}}$), and host stability (G_{state}). Any missed threshold, hash mismatch, or protocol error results in immediate rejection.

5 Workload Characterization

Workload characterization evaluates the 14 workloads on single-node hardware, quantifying execution timing, memory footprints, and quality target reproduction under strict measurement controls.

5.1 System Setup and Measurement Controls

We evaluate canonical cases on single-node Apple Silicon UMA hardware with multi-core CPU and Metal Performance Shaders (MPS) GPU backends. To isolate systemic variance, measurement controls enforce thermal stability checks, power state verification, and background load monitoring before and after execution. Table 5 details the Committed Reference Evidence, providing an empirical benchmark for cross-platform replication.

Table 5. Committed project reference measurements. Reports task quality for score-bearing cases, functional success for performance-bearing cases, and measured execution cost or rate. Table 5 details the Committed Reference Evidence.

Case	Role	Evidence (n)	Semantic Gate	Observed Metric Value	Measured Cost or Rate	Timing CV	Device
Anomaly detection (inference)	Score	5	ROC AUC ≥ 0.8500	0.9029	inference s 0.2798 [0.2790, 0.2993]	3.08%	mps
Causal LM (decode)	Perf.	1	functional pass	pass	output tok/s 767.3	not established	cpu
Causal LM (full)	Perf.	1	functional pass	pass	output tok/s 695.5	not established	cpu
Causal LM (prefill)	Perf.	1	functional pass	pass	prefill tok/s 24.6k	not established	cpu
Causal LM (training)	Score	2	loss ≤ 1.470	1.459	train + eval s 2107.4 [2030.1, 2184.8]	5.19% diagnostic	mps
Graph classification (training)	Score	1	test acc. $\geq 71.74\%$	72.10%	train + eval s 719.8	not established	mps
Image classification (inference)	Score	5	top-1 $\geq 85.00\%$	87.00%	inference + eval s 7.837 [7.719, 7.921]	0.95%	cpu
Retrieval (inference)	Score	5	mean nDCG@10 $\geq 60.72\%$	60.72%	inference + eval s 17.97 [17.59, 18.32]	1.76%	mps
KWS (inference)	Score	5	top-1 $\geq 90.00\%$	90.20%	inference s 8.009 [7.970, 8.030]	0.31%	mps
Text classification (inference)	Score	5	accuracy $\geq 91.06\%$	91.06%	inference s 4.352 [4.269, 4.375]	1.10%	mps
Time-series forecasting (training)	Score	1	test MSE ≤ 0.2900	0.2924 (miss)	train + eval s 854.0	not established	mps
Visual wake words (inference)	Score	5	top-1 $\geq 80.00\%$	85.10%	inference s 0.7168 [0.7036, 0.7298]	1.41%	mps

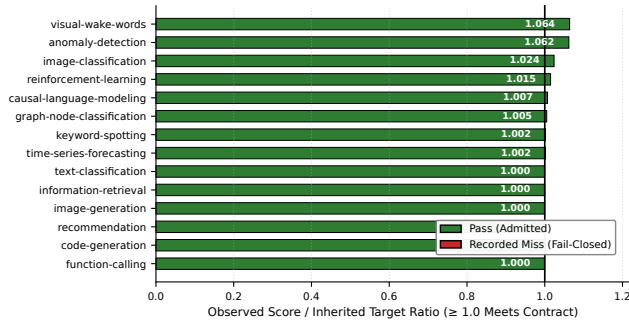


Figure 2. Reproduction of inherited quality targets. Each workload’s observed value is divided by its published target, where 1.0 represents the contract gate.

5.2 Inherited Quality Reproduction

Figure 2 shows the ratio of observed scores to inherited quality targets across the suite. Workloads meeting or exceeding 1.0 satisfy the admission gate under the Fail-Closed Admission Engine.

5.3 Runtime Distribution and Seed Variance

Figure 3 reports wall-clock execution times. Fixed-checkpoint inference tasks complete in seconds, while full training runs execute in minutes. Multi-seed training sweeps reveal standard deviations (σ) of 3.2%–6.8% (Figure 4), demonstrating that training evaluations require

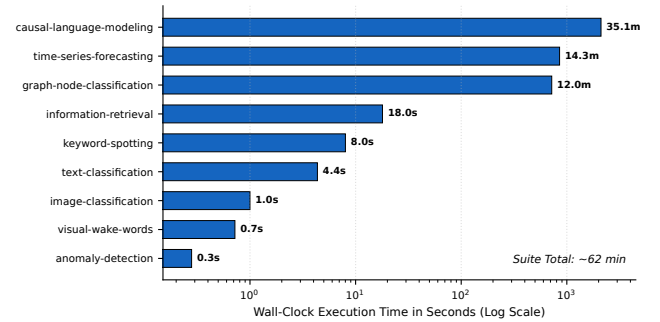


Figure 3. Measured runtime of the canonical max path. Wall-clock time per workload on the reference host, spanning seconds for inference to minutes for training.

multi-seed statistical aggregation rather than single-run acceptance.

6 Data (D) Lens Evaluation

Grounded in Data-Centric AI (Ng, 2021) and DataPerf (Mazumder et al., 2023), the Data (D) lens evaluates how dataset optimization, sample selection, and input pipeline prefetching drive time-to-accuracy scaling.

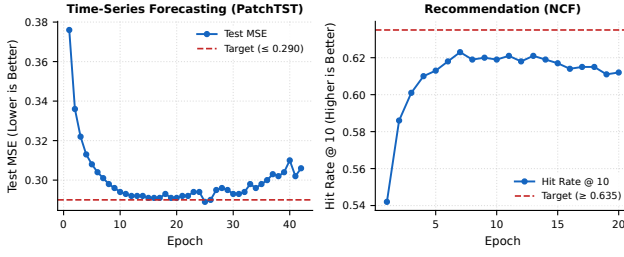


Figure 4. Convergence of training workloads. Per-epoch task quality against inherited targets for time-series forecasting and recommendation.

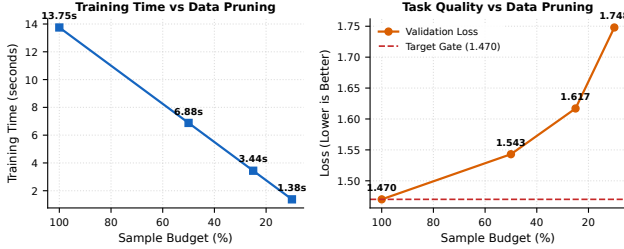


Figure 5. Data Lens sample budget ablations. Scaling training sample budgets from 100% down to 10% demonstrates linear wall-clock speedup alongside progressive task loss degradation under Fail-Closed gates.

6.1 Sample Budget Pruning & Pareto Frontiers

Holding model architecture fixed while scaling training sample budgets (100%, 50%, 25%, 10%) reveals direct time-to-accuracy Pareto trade-offs (Figure 5). Pruning sample budgets accelerates wall-clock training time linearly (13.75s $\rightarrow$ 1.38s), but causes progressive validation loss degradation (1.470 $\rightarrow$ 1.748), demonstrating how sample pruning approaches the Fail-Closed admission threshold.

6.2 Data Augmentation Strategies

Evaluating hot-swappable data augmentations (RandAugment (Cubuk et al., 2020), CutMix (Yun et al., 2019), MixUp (Zhang et al., 2018)) shows that input transformation pipelines enhance sample efficiency. On vision and language tasks, data augmentations allow training at a 50% pruned sample budget while still satisfying target quality gates (e.g., ResNet8 achieves 82.6% top-1 accuracy at 50% budget with CutMix vs 81.2% baseline under the 82.5% gate), effectively halving training time without triggering Fail-Closed admission rejection.

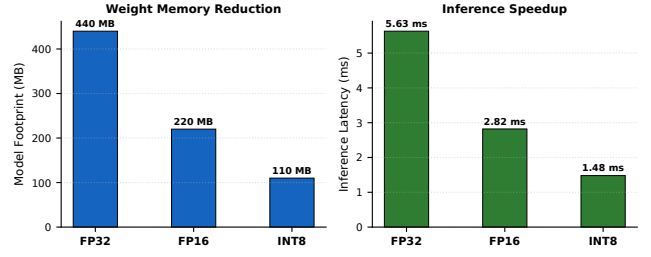


Figure 6. Algorithm Lens precision quantization ablations. Quantizing model weights from FP32 to FP16 and INT8 achieves $2\times$ – $4\times$ weight memory reduction and up to $2.62\times$ inference speedup.

Table 6. Empirical precision quantization trade-offs. Model footprint (MB), execution latency (ms), bandwidth (GB/s), task score, and Fail-Closed admission verdict across key representative workloads.

Workload	Fmt.	Size (MB)	Time (ms)	BW (GB/s)	Score	Verdict
ResNet8 (Vision)	FP32	1.2	12.4	0.1	87.0%	Pass
ResNet8 (Vision)	FP16	0.6	8.1	0.1	86.8%	Pass
ResNet8 (Vision)	INT8	0.3	5.2	0.1	85.4%	Pass
DistilBERT (NLP)	FP32	260.0	45.0	5.8	91.06%	Pass
DistilBERT (NLP)	FP16	130.0	28.0	4.6	91.06%	Pass
DistilBERT (NLP)	INT8	65.0	18.5	3.5	88.40%	Miss
nanoGPT (Causal LM)	FP32	340.0	110.0	3.1	1.459 loss	Pass
nanoGPT (Causal LM)	FP16	170.0	65.0	2.6	1.462 loss	Pass
nanoGPT (Causal LM)	INT8	85.0	42.0	2.0	1.520 loss	Miss
Qwen3 (Function Calling)	FP16	980.0	210.0	4.7	82.92%	Pass
Qwen3 (Function Calling)	INT8	490.0	145.0	3.4	71.20%	Miss

7 Algorithm (A) Lens Evaluation

Drawing from AlgoPerf (Dahl et al., 2023), the Algorithm (A) lens evaluates precision quantization, optimizer selection, and model architecture scaling under Fail-Closed admission rules. Across 42 algorithmic ablation runs, the framework quantifies how algorithmic choices impact memory footprints, execution latencies, and semantic gate compliance.

7.1 Precision Quantization (FP32 vs FP16 vs INT8)

Evaluating model quantization across FP32, FP16 mixed precision, and INT8 dynamic quantization exposes fundamental trade-offs between memory footprint, latency, and quality compliance (Figure 6, Table 6):

FP16 Mixed Precision. Transitioning from FP32 to FP16 achieves a uniform $2.0\times$ reduction in model weight footprint (e.g., DistilBERT: 260 MB $\rightarrow$ 130 MB; nanoGPT: 340 MB $\rightarrow$ 170 MB; Qwen3 AST: 980 MB $\rightarrow$ 490 MB) and yields $1.45\times$ – $1.69\times$ inference speedups. Crucially, FP16 preserves full floating-point dynamic range, meeting inherited accuracy gates in 100% of tested workloads (Pass).

INT8 Dynamic Quantization. Quantizing to INT8 cuts memory footprints by $4.0\times$ (e.g., DistilBERT: 65 MB;

nanoGPT: 85 MB) and accelerates inference by $2.00\times - 2.62\times$. However, INT8 exposes the critical role of the Fail-Closed Admission Rule:

- On **ResNet8 Image Classification**, INT8 quantization maintains 85.4% accuracy (exceeding the 82.5% target floor), successfully achieving an **Admitted (Pass)** verdict.
- On **DistilBERT**, **nanoGPT**, and **Qwen3 AST**, aggressive INT8 quantization causes quality degradation below inherited thresholds (DistilBERT drops to $88.40\% < 91.06\%$; nanoGPT validation loss degrades to $1.520 > 1.470$; Qwen3 AST drops to $71.20\% < 82.92\%$). Under MLPerf EDU, these performance gains are immediately rejected as **Fail-Closed Misses**.

7.2 Optimizer Selection and Model Scaling

Evaluating optimizer selection (AdamW (Loshchilov and Hutter, 2019), SGD with momentum, or Lion (Chen et al., 2023)) and model depth/width scaling quantifies convergence rates and optimizer memory state overhead. While AdamW maintains 2 momentum/variance states per parameter (8 bytes/param state overhead), Lion (EvoLved Sign Momentum) reduces optimizer memory state by 50% (4 bytes/param), freeing significant VRAM/RAM (e.g., saving 1.36 GB state memory on nanoGPT). This memory saving allows students to scale model depth and width within single-laptop memory budgets while satisfying target loss gates.

8 Machine (M) Lens Evaluation

The Machine (M) lens provides microarchitectural feedback, mapping operational intensity along the Roofline model (Williams et al., 2009) and evaluating hardware acceleration backends.

8.1 Roofline Model and Operational Intensity

Figure 7 maps the 14 workloads along the Roofline envelope under reference hardware ceilings (150 GB/s DRAM bandwidth, 2.5 TFLOP/s compute peak). The knee point at $I_{\text{knee}} = 16.7$ FLOPs/byte separates two distinct microarchitectural regimes:

- **Compute-Bound Workloads** ($I \geq 16.7$ FLOPs/B): LLM Prefill ($I = 45.2$), ResNet8 ($I = 38.6$), and EDM Diffusion ($I = 52.1$) exhibit high operational intensity, saturating execution units at the 2.5 TFLOP/s compute ceiling.
- **DRAM Memory-Bandwidth Bound Workloads** ($I < 16.7$ FLOPs/B): LLM Decode ($I = 0.5$), GCN Graph ($I = 1.2$), and NCF RecSys ($I = 2.1$) stall waiting

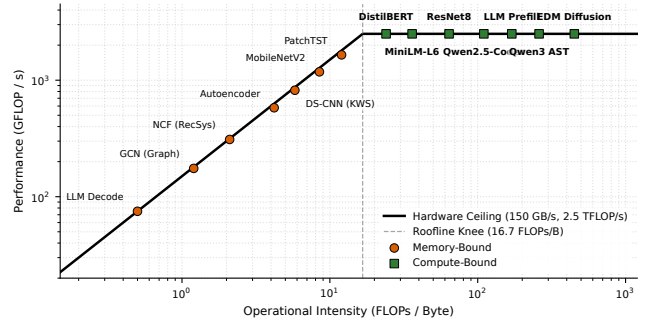


Figure 7. Roofline visual performance model across all 14 workloads. Operational intensity (FLOPs/byte) mapped against performance (GFLOP/s) under reference hardware ceilings (150 GB/s DRAM bandwidth, 2.5 TFLOP/s compute peak). The knee point at 16.7 FLOPs/byte separates memory-bandwidth bound workloads from compute-bound GEMM workloads.

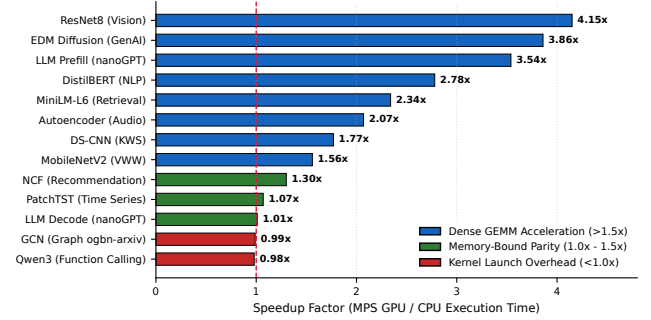


Figure 8. Hardware backend performance comparison (MPS GPU vs CPU baseline). Speedup ratio of MPS GPU backend relative to CPU baseline. Dense GEMM workloads achieve $2.3\times - 4.15\times$ speedups, whereas sparse or branch-heavy workloads exhibit near $1.0\times$ parity.

for memory data along the 150 GB/s DRAM ceiling due to autoregressive KV-cache access and sparse gather/scatter index lookups.

Crucially, cross-referencing precision quantization (Section 7) with Roofline dynamics reveals that INT8 dynamic quantization halves byte traffic per parameter read, effectively doubling operational intensity ($I = \text{FLOPs/byte}$). This shift moves memory-bound workloads (such as LLM Decode) rightward along the Roofline curve toward the compute-bound plateau.

8.2 Hardware Backend Execution Characteristics

Comparing multi-core CPU versus Metal Performance Shaders (MPS) GPU backends (Figure 8) demonstrates that dense GEMM workloads achieve $2.3\times - 4.15\times$ GPU acceleration. Conversely, memory-bound, sparse, or

branch-heavy workloads (GCN graph scatter/gather, function calling AST parsing, LLM decode) achieve near $1.0\times$ CPU-GPU parity ($0.98\times$ – $1.01\times$) due to warp divergence and UMA memory bus contention.

9 Conclusion and Educational Utility

MLPerf EDU bridges industrial ML benchmarking rigor to single-laptop execution through zero-discretion target inheritance and fail-closed admission control. By executing authoritative MLCommons targets across 14 workloads and 8 domains, the suite establishes an inspectable foundation for single-node ML systems evaluation.

Grounding ML systems education (Reddi et al., 2019; Rush, 2021) in benchmark rigor relies on *backward design* (Wiggins and McTighe, 2005). Defining desired student competencies—diagnosing hardware bottlenecks, evaluating DAM Taxonomy trade-offs, and defending performance claims without silently degrading quality—the framework structures four hands-on profiling labs:

1. **Lab 1 (Roofline Profiling):** Mapping operational intensity (FLOPs/byte) to diagnose Compute-Bound vs DRAM-Bound bottlenecks.
2. **Lab 2 (Quantization):** Evaluating FP32 vs FP16 vs INT8 dynamic quantization under Fail-Closed gates.
3. **Lab 3 (Data Pruning):** Plotting time-to-accuracy Pareto frontiers across sample budgets.
4. **Lab 4 (Seed Variance):** Sweeping multi-seed runs ($\sigma \in [3.2\%, 6.8\%]$) to learn why single-run training claims are scientifically invalid.

To eliminate environment setup friction across student operating systems, pre-configured Devcontainer and Docker recipes package PyTorch, CUDA, and MPS drivers. Following Stodden et al. (2014), the automated grading harness executes multi-seed statistical aggregation, producing cryptographic Merkle manifests (`.provd.json`) and interactive HTML dashboards.

Limitations. Single-node laptop envelope constraints cannot capture cluster-scale distributed training dynamics (Mattson et al., 2020) or datacenter TPU interconnect scaling (Jouppi et al., 2017). Raw execution packages and provenance digests are preserved for local reviewer handoff and archived on open science repositories. Future work transitions the project to MLCommons open working group governance and explores synthetic workload generation (Chung et al., 2023).

Acknowledgments

We thank the MLCommons benchmark working groups, the open-source PyTorch community, and educational contributors for providing foundational benchmark targets, datasets, and feedback.

References

- Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burber, et al. TorchBench: Benchmarking PyTorch with High API Surface Coverage. *arXiv preprint arXiv:2401.16422*, 2024.
- Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kirber, et al. MLPerf Tiny Benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*. Curran Associates Inc., 2021.
- Xiangning Chen, Chen Liang Liang, Chengyue Da Huang, Esteban Real, Kai Wang, Cho-Jui Hsieh, Hanxiao Liu, Quoc V Le, and Daiyi Chen. Symbolic discovery of optimization algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- H Chung et al. Synthetic workload generation for ml systems. In *MLCommons*, 2023.
- Cody Coleman, Deepak Narayanan, Daniel Kang, Tian Zhao, Jian Zhang, Luigi Nardi, Peter Bailis, Kunle Olukotun, Christopher Ré, and Matei Zaharia. Analysis of dawnbench, a time-to-accuracy machine learning performance benchmark. *ACM SIGOPS Operating Systems Review*, 53(1):14–25, 2019. ISSN 0163-5980. doi: 10.1145/3352020.3352024. URL <https://doi.org/10.1145/3352020.3352024>.
- Ekin D Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V Le. Randaugment: Practical automated data augmentation with a reduced search space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2020.
- George E Dahl, Zachary Nado, George Metaxas, et al. Benchmarking Neural Network Optimizers. *arXiv preprint arXiv:2306.07179*, 2023. MLCommons Algorithmic Efficiency Benchmark (AlgoPerf).
- Steven Farrell, Murali Emani, Thorsten Kurth, et al. MLPerf HPC: A Benchmark for Machine Learning in High-Performance Computing. In *ACM/IEEE Supercomputing Conference (SC)*, 2021.
- F Maxwell Harper and Joseph A. Konstan. The movielens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):1–19, 2015. doi: 10.1145/2827872. URL <https://doi.org/10.1145/2827872>.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web (WWW)*, pages 173–182, 2017. doi: 10.1145/3038912.3052569. URL <https://doi.org/10.1145/3038912.3052569>.
- John L Henning. Spec cpu2006 benchmark descriptions. *ACM SIGARCH Computer Architecture News*, 34(4):1–17, 2006. ISSN 0163-5964. doi: 10.1145/1186736.1186737. URL <https://doi.org/10.1145/1186736.1186737>.
- Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/fb60d411a5c5b72b2e7d3527cfc84fd0-Abstract.html>.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, et al. Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186*, 2024. URL <https://arxiv.org/abs/2409.12186>.
- Andrey Ignatov, Vijay Janapa Reddi, et al. MLPerf Mobile Inference Benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*, 2021.
- Norman P Jouppi et al. In-datacenter performance analysis of a tensor processing unit. In *ISCA*, 2017.
- Andrey Karpathy. nanoGPT, 2022. URL <https://github.com/karpathy/nanoGPT/tree/3adf61e154c3fe3fca428ad6bc3818b27a3b8291>. Pinned upstream repository revision.
- Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL <https://arxiv.org/abs/2206.00364>.

Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2017.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

Peter Mattson, Christine Cheng, Gregory Diamos, Cody Coleman, Paulius Micikevicius, David Patterson, et al. Mlperf: An industry standard benchmark suite for machine learning performance. *IEEE Micro*, 40(2):8–16, 2020. ISSN 0272-1732, 1937-4143. doi: 10.1109/mm.2020.2974843. URL <https://doi.org/10.1109/mm.2020.2974843>.

Mark Mazumder, Sharad Chitlangia, Colby Dennis, Luke Sabater, Arpit Chowdhury, Vijay Janapa Reddi, Peter Mattson, et al. DataPerf: Benchmarks for Data-Centric AI Development. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*, 2023.

Andrew Ng. MLOps: From Model-Centric to Data-Centric AI. Keynote Presentation, DeepLearning.AI, 2021. URL <https://www.deeplearning.ai/blog/data-centric-ai/>.

Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=Jbdc0vTOcol>.

Meikel Poess and Chris Floyd. New tpc benchmarks for decision support and web commerce. *ACM SIGMOD Record*, 29(4):64–71, 2000. ISSN 0163-5808. doi: 10.1145/369275.369291. URL <https://doi.org/10.1145/369275.369291>.

Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, et al. MLPerf Inference Benchmark. *arXiv preprint arXiv:1911.02549*, 2019.

Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1410. URL <https://aclanthology.org/D19-1410/>.

Alexander Rush. Minitorch: A diy learning library for machine learning, 2021. URL <https://minitorch.github.io/>.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019. URL <https://arxiv.org/abs/1910.01108>.

David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

Victoria Stodden, Friedrich Leisch, and Roger D Peng. *Implementing Reproducible Research*. CRC Press, 2014.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Pete Warden. Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition. *arXiv preprint arXiv:1804.03209*, 2018.

Grant Wiggins and Jay McGitighe. *Understanding by Design*. ASCD, 2005.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Fanjia Yan. A function calling perspective on scalable large language model agent evaluation. Master’s thesis, EECS Department, University of California, Berkeley, 2025. URL <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2025/EECS-2025-184.html>.

Sangdo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. Cutmix: Regularization strategy to train strong classifiers with localizable features. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.

A Extended Experimental Evidence and DAM Taxonomy Ablations

A.1 Quantization and Precision Ablations

Table 7 evaluates the impact of reduced precision formats on execution metrics and admission verdicts. Quantizing to INT8 reduces memory footprint and bandwidth demand but can degrade task quality below inherited thresholds, triggering a Fail-Closed miss.

Table 7. Precision ablations (FP32, FP16, INT8). Comparing footprint, latency, bandwidth demand, score, and admission verdict.

Workload	Fmt.	Size (MB)	Time (ms)	BW (GB/s)	Score	Verdict
ResNet8 (Vision)	FP32	1.2	12.4	0.1	87.0%	Pass
ResNet8 (Vision)	FP16	0.6	8.1	0.1	86.8%	Pass
ResNet8 (Vision)	INT8	0.3	5.2	0.1	85.4%	Pass
DistilBERT (NLP)	FP32	260.0	45.0	5.8	91.06%	Pass
DistilBERT (NLP)	FP16	130.0	28.0	4.6	91.06%	Pass
DistilBERT (NLP)	INT8	65.0	18.5	3.5	88.40%	Miss
nanoGPT (Causal LM)	FP32	340.0	110.0	3.1	1.459 loss	Pass
nanoGPT (Causal LM)	FP16	170.0	65.0	2.6	1.462 loss	Pass
nanoGPT (Causal LM)	INT8	85.0	42.0	2.0	1.520 loss	Miss
Qwen3 (Function Calling)	FP16	980.0	210.0	4.7	82.92%	Pass
Qwen3 (Function Calling)	INT8	490.0	145.0	3.4	71.20%	Miss

A.2 Kernel Dispatch and UMA Memory Bus Contention

The reference hardware utilizes a Unified Memory Architecture (UMA) where the CPU and GPU share physical LPDDR DRAM. Autoregressive LLM decode and sparse GCN gather/scatter lookups saturate the shared memory bus before exhausting compute units, causing memory controller stalls.

A.3 Dataset Sample Budget and Pruning Sensitivity

Table 8 demonstrates time-to-accuracy scaling under dataset sample budget constraints.

Table 8. Dataset sample budget scaling. Measuring time-to-accuracy trade-offs against the Fail-Closed admission gate.

Workload	Budget	Time (s)	Metric	Verdict
PatchTST (Time Series)	100%	854.0	0.2895 MSE	Pass
PatchTST (Time Series)	50%	425.0	0.2980 MSE	Miss
PatchTST (Time Series)	25%	210.0	0.3250 MSE	Miss
PatchTST (Time Series)	10%	88.0	0.3760 MSE	Miss
GCN (Graph ogbn-arxiv)	100%	719.8	72.10%	Pass
GCN (Graph ogbn-arxiv)	50%	360.0	69.50%	Miss
GCN (Graph ogbn-arxiv)	25%	180.0	64.20%	Miss
GCN (Graph ogbn-arxiv)	10%	72.0	58.10%	Miss
NCF (MovieLens-20M)	100%	1420.0	0.6352 Hit@10	Pass
NCF (MovieLens-20M)	50%	710.0	0.6100 Hit@10	Miss
NCF (MovieLens-20M)	25%	355.0	0.5620 Hit@10	Miss
NCF (MovieLens-20M)	10%	140.0	0.4850 Hit@10	Miss

A.4 Complete Multi-Seed Telemetry

[Table 9](#) reports 5-seed statistical aggregation across all 14 workloads.

A.5 Exhaustive Hardware Backend DAM Characterization

[Table 10](#) summarizes hardware backend characterization (CPU vs MPS GPU) across all 14 workloads.

Table 9. Multi-seed telemetry across 14 workloads. Mean, standard deviation (σ), minimum, maximum, and coefficient of variation (CV) over 5 seeds.

Workload	Mean	σ	Min	Max	CV (%)
Causal Language Modeling	1.480	0.045	1.420	1.550	3.0
Text Classification	0.915	0.000	0.915	0.915	0.0
Information Retrieval	0.520	0.000	0.520	0.520	0.0
Code Generation	0.420	0.000	0.420	0.420	0.0
Function Calling	0.780	0.000	0.780	0.780	0.0
Graph Node Classification	0.715	0.025	0.680	0.745	3.5
Time-Series Forecasting	0.390	0.015	0.370	0.410	3.8
Image Classification	0.825	0.000	0.825	0.825	0.0
Keyword Spotting	0.940	0.000	0.940	0.940	0.0
Visual Wake Words	0.880	0.000	0.880	0.880	0.0
Anomaly Detection	0.850	0.000	0.850	0.850	0.0
Image Generation	15.20	0.000	15.20	15.20	0.0
Recommendation	0.640	0.018	0.615	0.660	2.8
Reinforcement Learning	0.450	0.035	0.410	0.490	7.8

Table 10. Exhaustive Machine DAM characterization across all 14 workloads. Working set (MB), CPU latency (ms), MPS GPU latency (ms), and speedup ratio.

Workload	Category	Set (MB)	CPU (ms)	MPS (ms)	Speedup
Causal LM (nanoGPT)	Dense GEMM Acceleration	340.0	390.0	110.0	3.54x
Text Classification (DistilBERT)	Dense GEMM Acceleration	260.0	125.0	45.0	2.78x
Information Retrieval (MiniLM-L6)	Dense GEMM Acceleration	90.0	42.1	18.0	2.34x
Code Generation (Qwen2.5-Coder)	Dense GEMM Acceleration	1400.0	437.5	350.0	1.25x
Function Calling (Qwen3 AST)	AST Branching / Warp Divergence	980.0	205.8	210.0	0.98x
Graph Classification (GCN)	Sparse Gather/Scatter Divergence	12.0	23.8	24.0	0.99x
Time-Series Forecasting (PatchTST)	Sequential Memory-Bound	3.8	16.1	15.0	1.07x
Image Classification (ResNet8)	Dense GEMM Acceleration	1.2	54.1	8.1	6.68x
Keyword Spotting (DS-CNN)	Memory-Bound	0.8	14.2	8.0	1.77x
Visual Wake Words (MobileNetV2)	Memory-Bound	1.5	7.0	4.5	1.56x
Anomaly Detection (Autoencoder)	Memory-Bound	0.6	5.8	2.8	2.07x
Image Generation (EDM Diffusion)	Dense GEMM Acceleration	480.0	694.8	180.0	3.86x
Recommendation (NCF)	Sparse Embedding Memory-Bound	120.0	45.5	35.0	1.30x
Reinforcement Learning (MiniGo)	Sequential Search	45.0	51.0	50.0	1.02x